

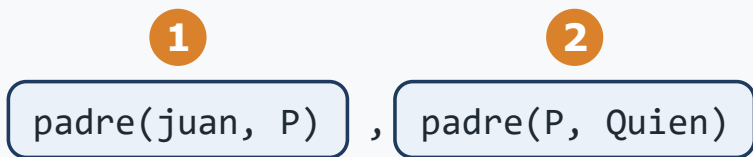
Capítulo 5 — Cómo responde

Prolog

Dos órdenes

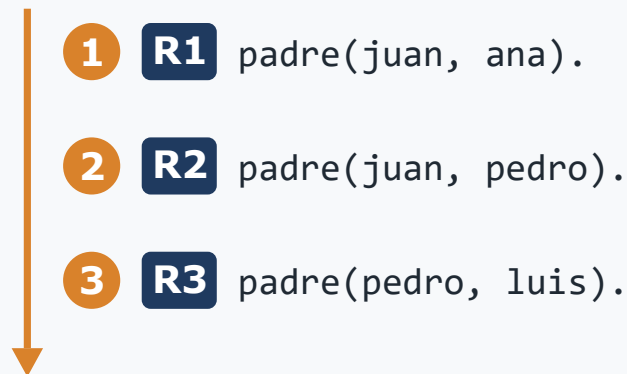
objetivos

de izquierda a derecha



cláusulas

de arriba hacia abajo

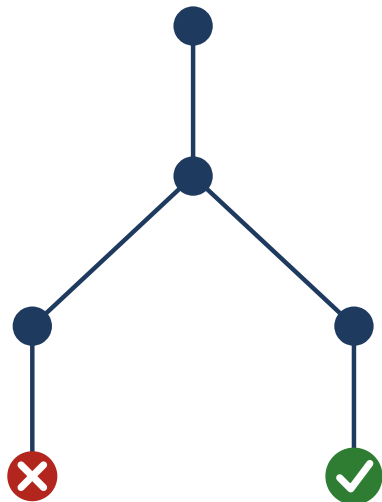


y si un objetivo falla:



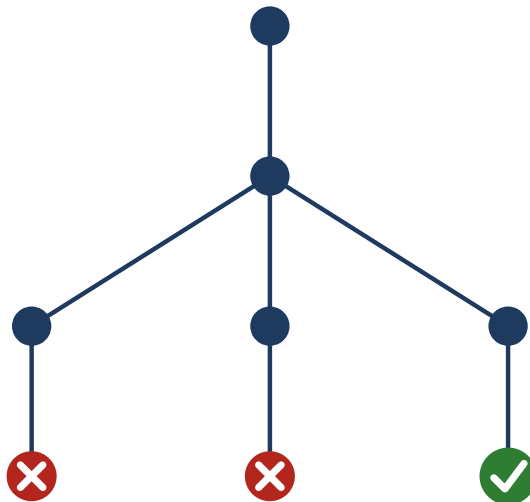
Qué cambia el orden de los objetivos

un orden



2 ramas

el otro orden

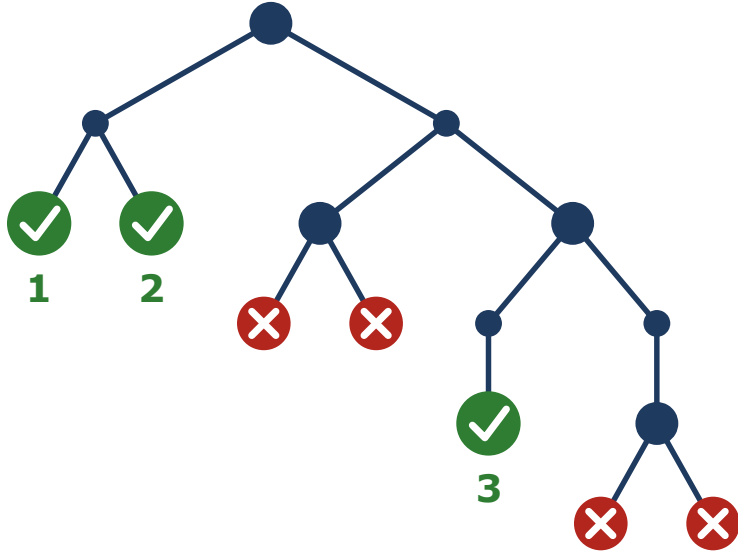


3 ramas

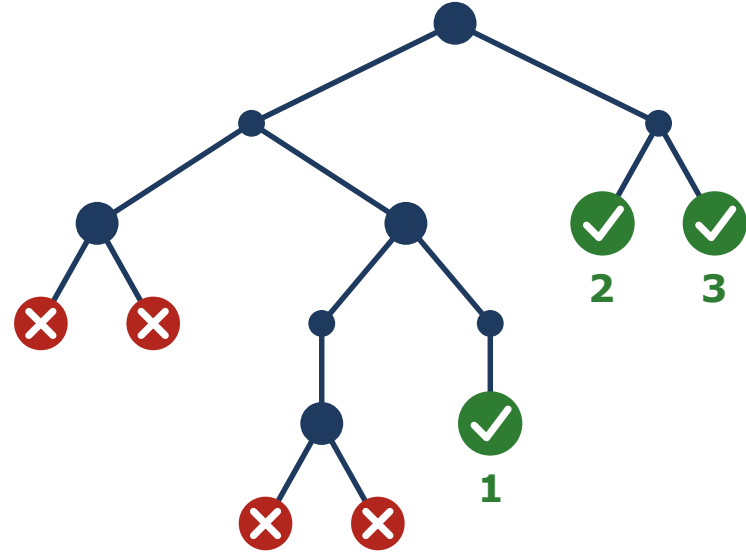
otro árbol: cambia el trabajo

Qué cambia el orden de las cláusulas

un orden



el orden inverso



el mismo árbol en espejo: cambia el orden de las respuestas

El programa

```
% padre(P, H): P es el padre de H.
```

```
padre(juan, ana).
```

```
padre(juan, pedro).
```

```
padre(pedro, luis).
```

```
%! abuelo(?A, ?N) is nondet.
```

```
%
```

```
% A es abuelo de N.
```

```
abuelo(A, N) :-
```

```
    padre(A, P),
```

```
    padre(P, N).
```

Las cláusulas, numeradas

R1 padre(juan, ana).

R2 padre(juan, pedro).

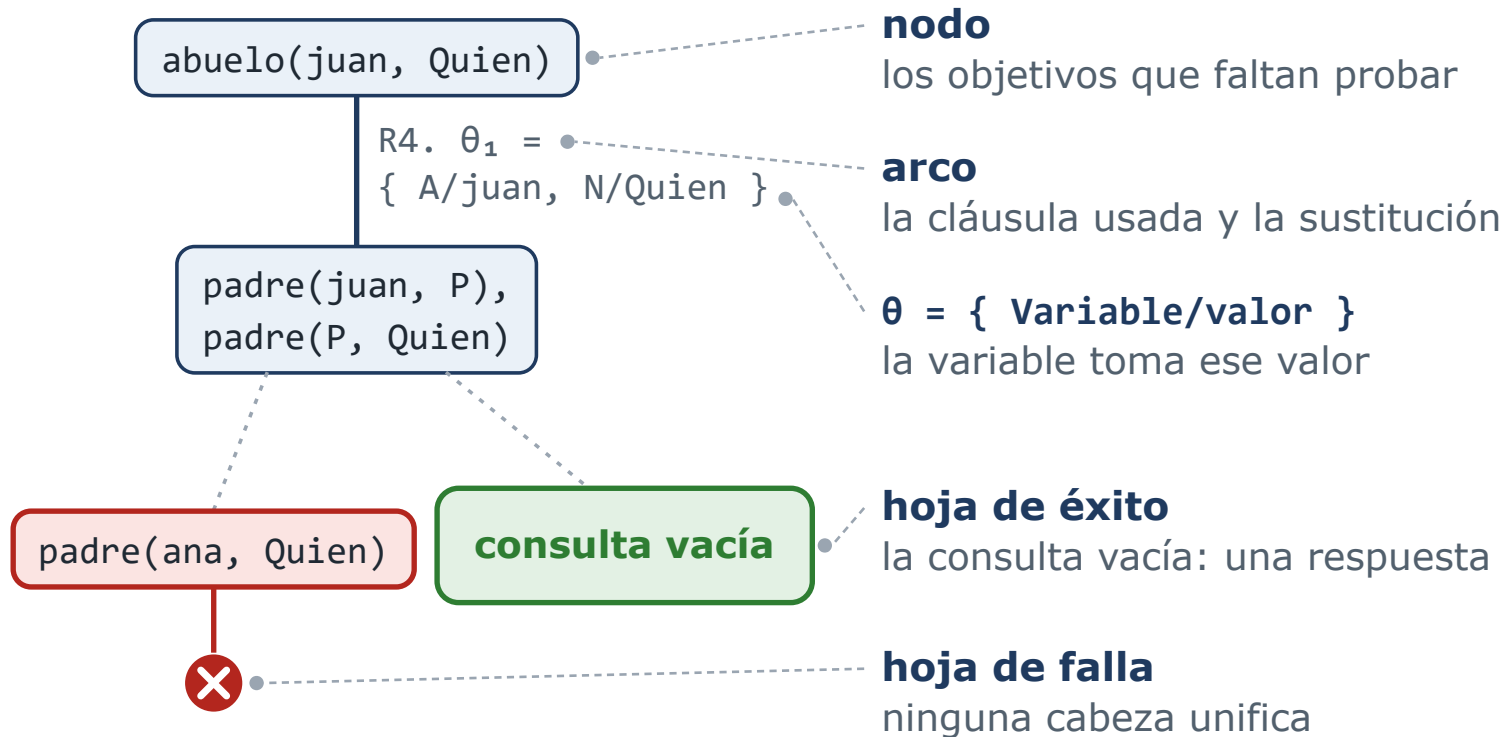
R3 padre(pedro, luis).

R4 abuelo(A, N) :-
 padre(A, P),
 padre(P, N).

consulta

?- abuelo(juan, Quien).

Las partes de un árbol



El árbol · 1

abuelo(juan, Quien)

programa

R1 padre(juan, ana).

R2 padre(juan, pedro).

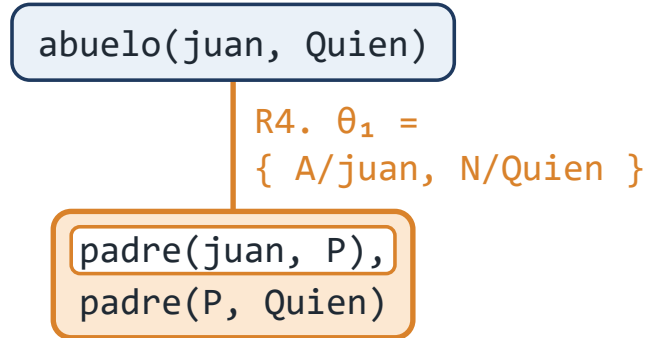
R3 padre(pedro, luis).

R4 abuelo(A, N) :-
 padre(A, P),
 padre(P, N).

La raíz

la consulta inicial:
los objetivos
por probar

El árbol - 2



programa

R1 `padre(juan, ana).`

R2 `padre(juan, pedro).`

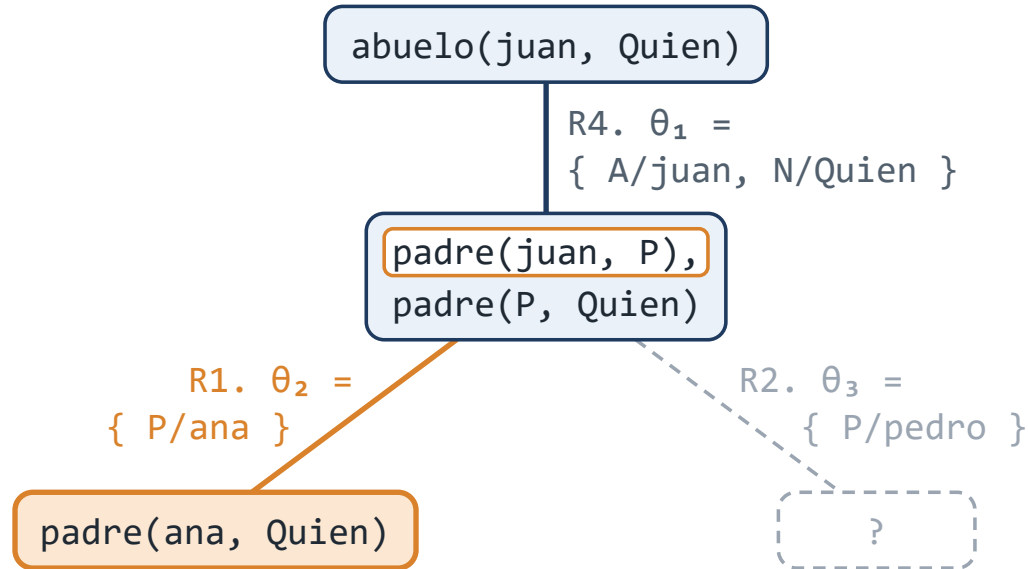
R3 `padre(pedro, luis).`

R4 `abuelo(A, N) :-
 padre(A, P),
 padre(P, N).`

Resolución con R4

el objetivo, cambiado
por el cuerpo de R4,
con θ_1 aplicada

El árbol - 3



programa

R1 `padre(juan, ana).`

R2 `padre(juan, pedro).`

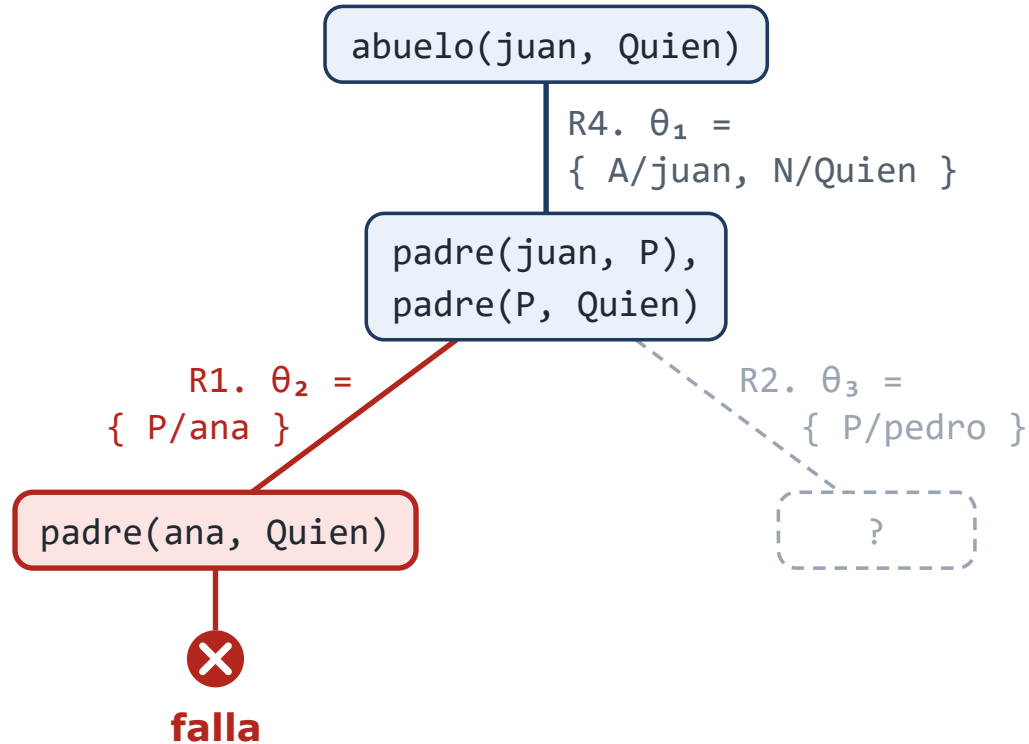
R3 `padre(pedro, luis).`

R4 `abuelo(A, N) :-
 padre(A, P),
 padre(P, N).`

Dos ramas

R1 y R2 unifican
con `padre(juan, P)`:
primero, la de R1;
R2, pendiente

El árbol - 4



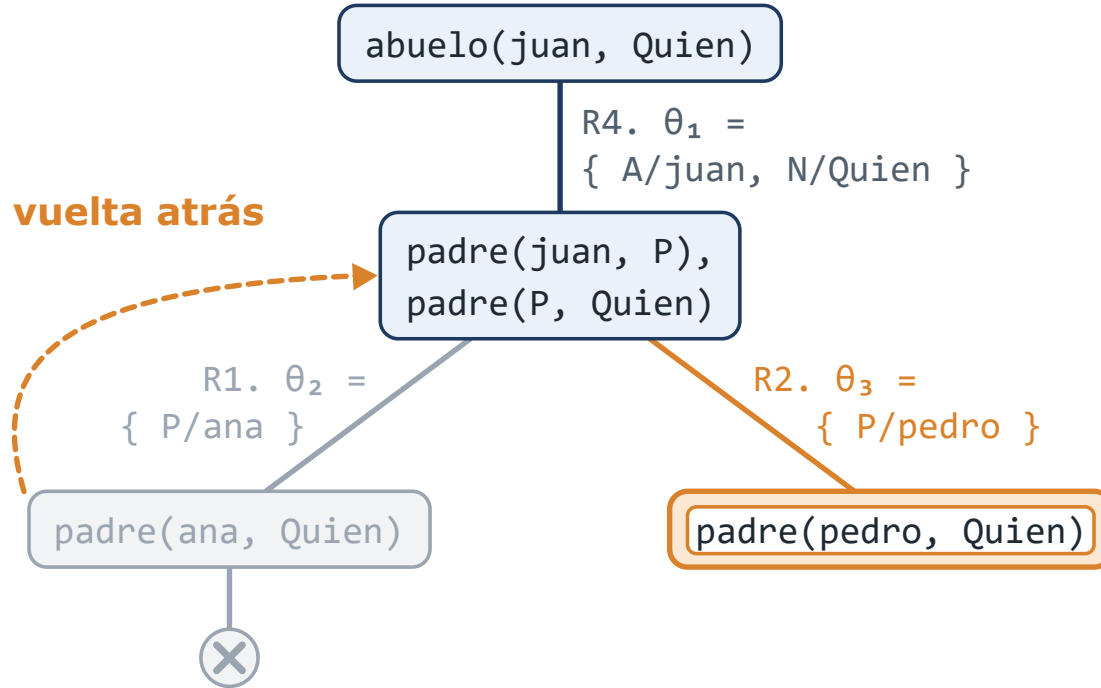
programa

- ✗ **R1** padre(juan, ana).
- ✗ **R2** padre(juan, pedro).
- ✗ **R3** padre(pedro, luis).
- R4** abuelo(A, N) :-
 padre(A, P),
 padre(P, N).

Falla

ninguna cabeza
unifica con
padre(ana, Quien)

El árbol - 5



programa

R1 padre(juan, ana).

R2 padre(juan, pedro).

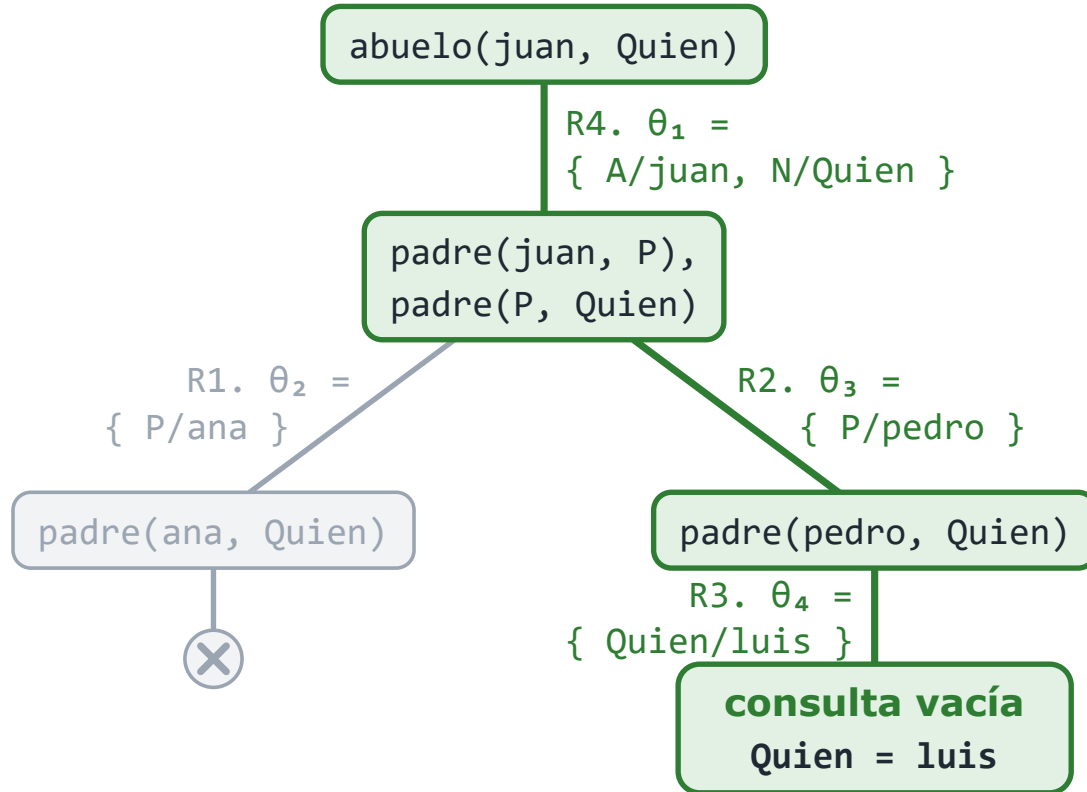
R3 padre(pedro, luis).

R4 abuelo(A, N) :-
 padre(A, P),
 padre(P, N).

Vuelta atrás

a la última decisión:
la alternativa
pendiente, R2

El árbol - 6



programa

R1 padre(juan, ana).

R2 padre(juan, pedro).

R3 padre(pedro, luis).

R4 abuelo(A, N) :-
 padre(A, P),
 padre(P, N).

Éxito

no queda ningún
objetivo: la
consulta vacía

Resolución

objetivo

abuelo(juan, Quien)

unifican

cabeza de R4

abuelo(A, N) :- ...

$\theta_1 = \{ A/\text{juan}, N/\text{Quien} \}$

- 1 se reemplaza el objetivo por el cuerpo de R4

padre(A, P), padre(P, N)

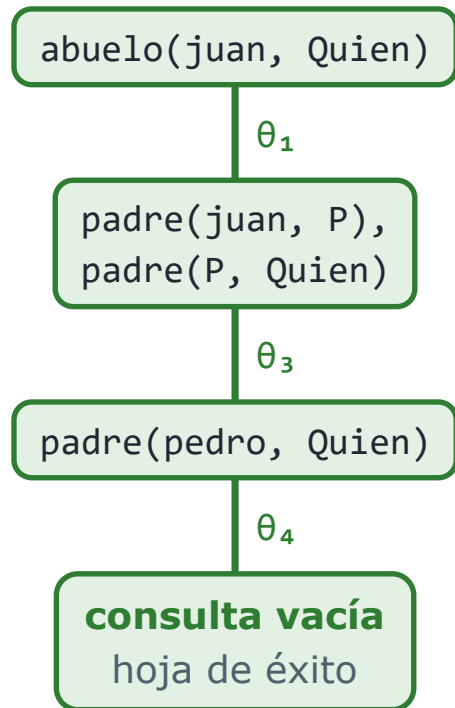


- 2 se aplica θ_1 a todos los objetivos que quedan

padre(juan, P), padre(P, Quien)

el nodo hijo del árbol

La respuesta



sustituciones de la rama

$\theta_1 = \{ A/\text{juan}, N/\text{Quien} \}$

$\theta_3 = \{ P/\text{pedro} \}$

$\theta_4 = \{ \text{Quien}/\text{luis} \}$

A, N, P:
variables de R4,
no aparecen

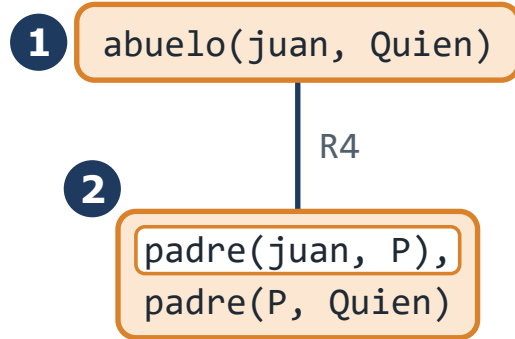
las variables de la consulta

Quien = luis.

La traza

```
Call: (10) abuelo(juan, _8106)
Call: (11) padre(juan, _8960)
Exit: (11) padre(juan, ana)
Call: (11) padre(ana, _8106)
Fail: (11) padre(ana, _8106)
Redo: (11) padre(juan, _8960)
Exit: (11) padre(juan, pedro)
Call: (11) padre(pedro, _8106)
Exit: (11) padre(pedro, luis)
Exit: (10) abuelo(juan, luis)
```

Traza y árbol · 1

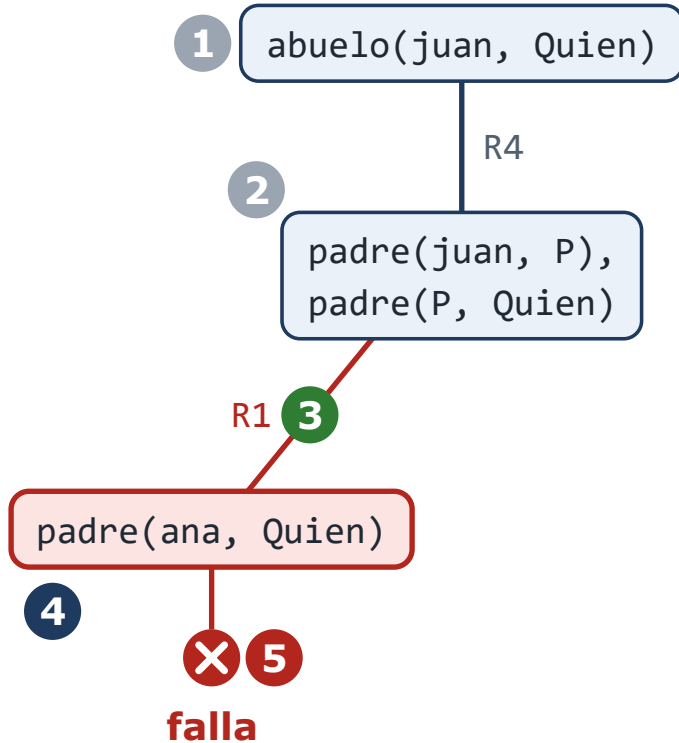


trace

1 Call: (10) `abuelo(juan, _8106)`

2 Call: (11) `padre(juan, _8960)`

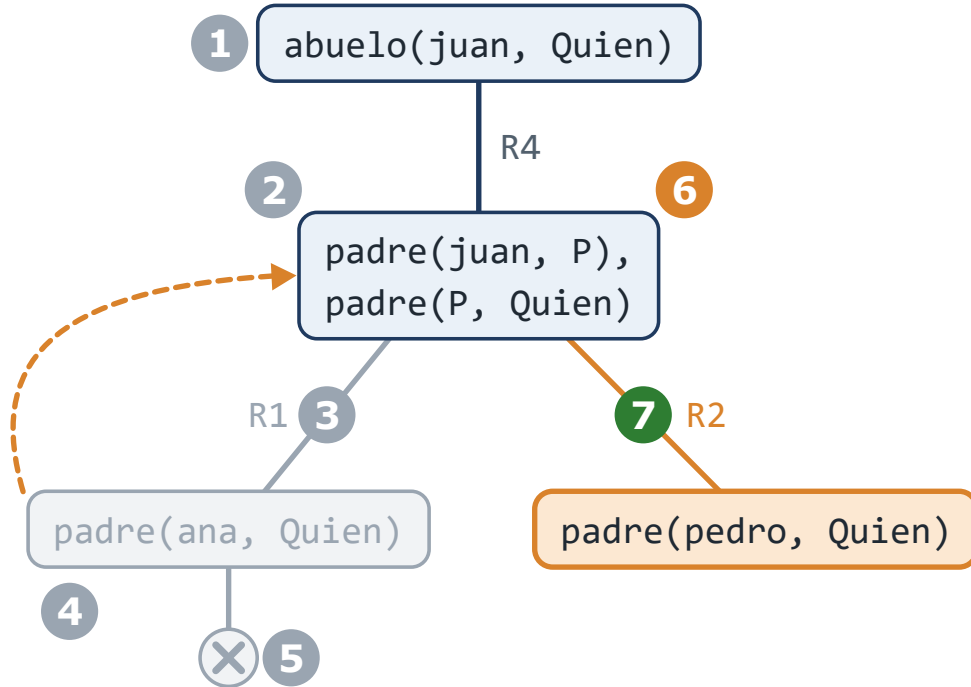
Traza y árbol · 2



trace

- 1 Call: (10) abuelo(juan, _8106)
- 2 Call: (11) padre(juan, _8960)
- 3 Exit: (11) padre(juan, ana)
- 4 Call: (11) padre(ana, _8106)
- 5 Fail: (11) padre(ana, _8106)

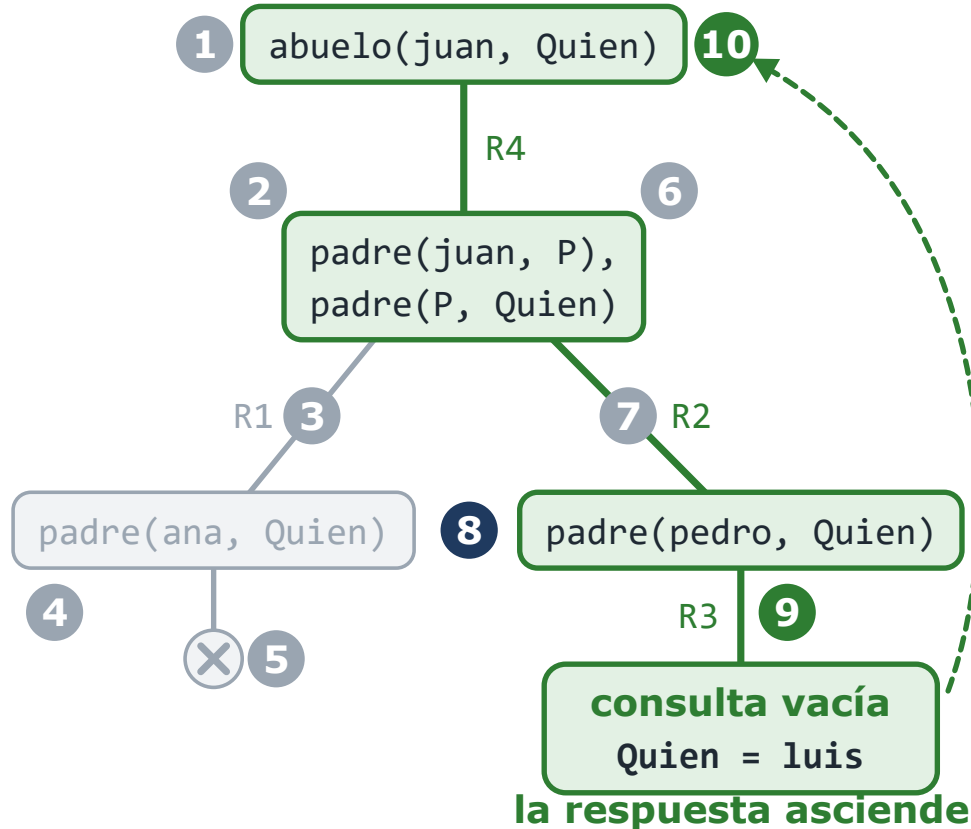
Traza y árbol - 3



trace

- 1 Call: (10) `abuelo(juan, _8106)`
- 2 Call: (11) `padre(juan, _8960)`
- 3 Exit: (11) `padre(juan, ana)`
- 4 Call: (11) `padre(ana, _8106)`
- 5 Fail: (11) `padre(ana, _8106)`
- 6 Redo: (11) `padre(juan, _8960)`
- 7 Exit: (11) `padre(juan, pedro)`

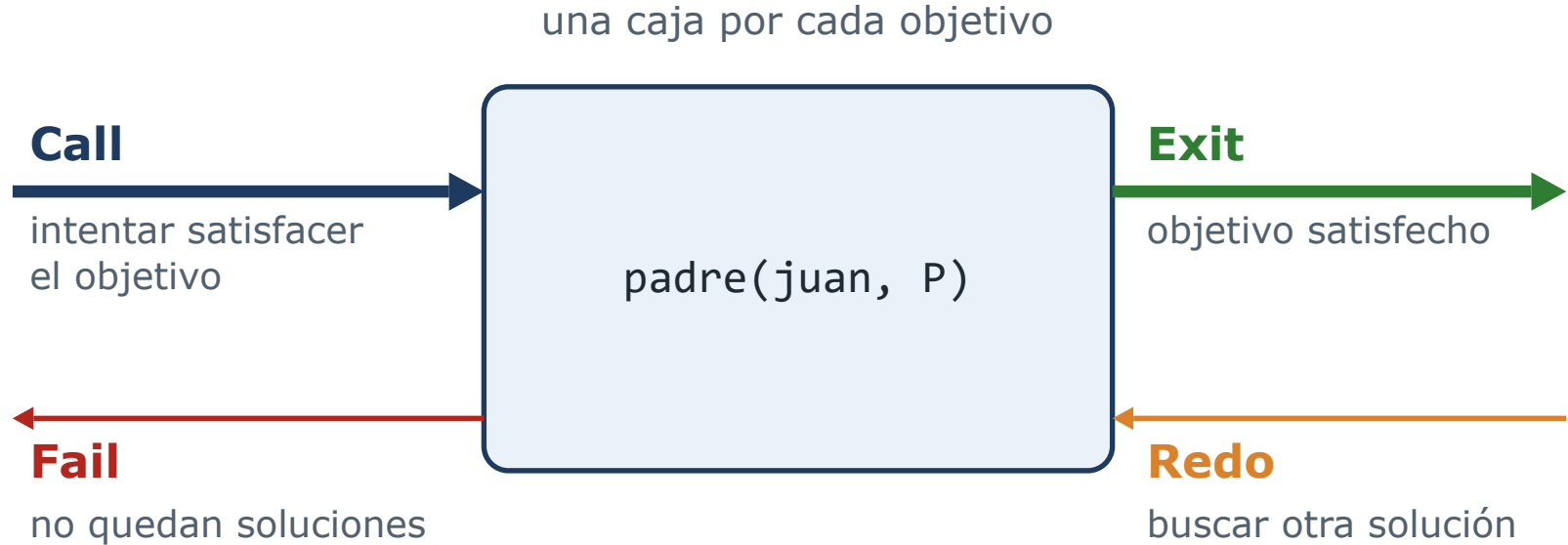
Traza y árbol · 4



trace

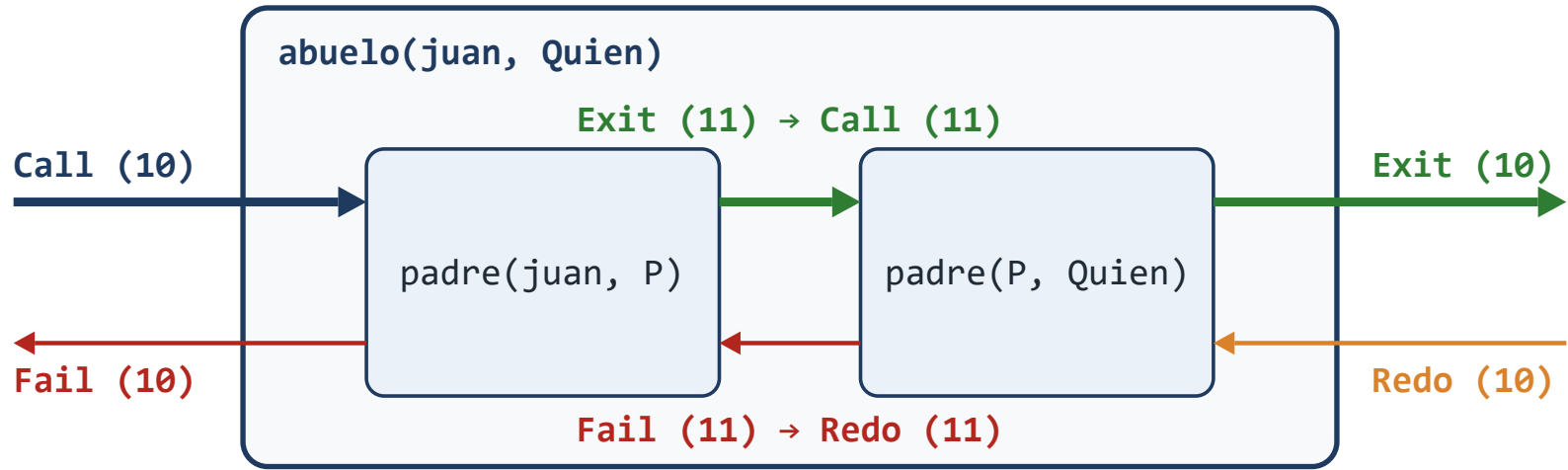
- 1 Call: (10) abuelo(juan, _8106)
- 2 Call: (11) padre(juan, _8960)
- 3 Exit: (11) padre(juan, ana)
- 4 Call: (11) padre(ana, _8106)
- 5 Fail: (11) padre(ana, _8106)
- 6 Redo: (11) padre(juan, _8960)
- 7 Exit: (11) padre(juan, pedro)
- 8 Call: (11) padre(pedro, _8106)
- 9 Exit: (11) padre(pedro, luis)
- 10 Exit: (10) abuelo(juan, luis)

Una caja por objetivo



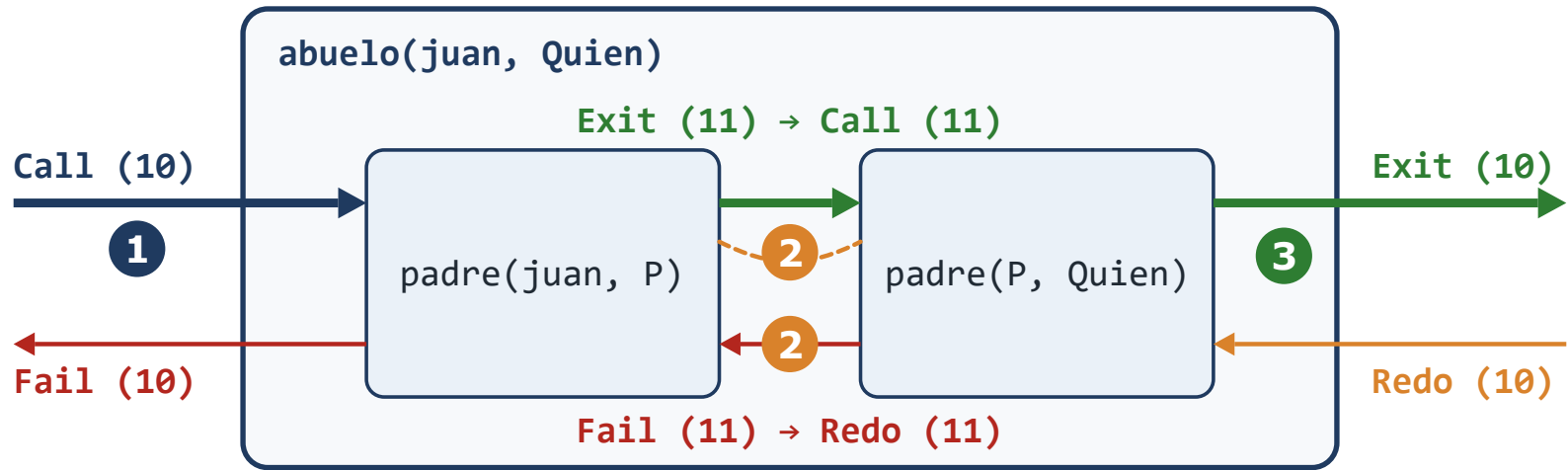
flechas gruesas: el avance · flechas finas: el retroceso

Cajas anidadas



el Exit de una caja lleva al Call de la siguiente;
el Fail de una caja lleva al Redo de la anterior

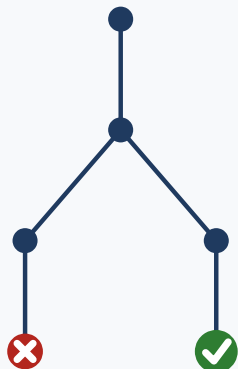
El recorrido por las cajas



- 1** Call (10): se entra a la primera caja
- 2** P = ana; padre(ana, Quien) falla: Redo en la primera caja
- 3** P = pedro; padre(pedro, Quien) da Quien = luis: Exit (10)

Tres diagramas

el árbol



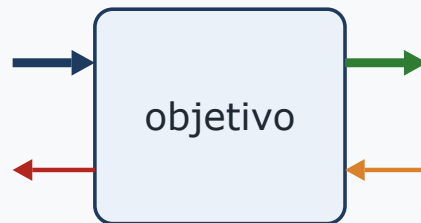
todas las
alternativas, también
las no recorridas

la traza

```
Call: (10) abuelo(...)
Call: (11) padre(...)
Exit: (11) padre(...)
Call: (11) padre(...)
Fail: (11) padre(...)
Redo: (11) padre(...)
Exit: (11) padre(...)
Call: (11) padre(...)
Exit: (11) padre(...)
Exit: (10) abuelo(...)
```

el orden exacto
de los pasos

las cajas



por dónde entra
y sale la ejecución
de un objetivo

Caso base primero

```
%! antepasado(?A, ?D) is nondet.
```

```
%
```

```
% A es antepasado de D, con el caso base escrito primero.
```

```
antepasado(A, D) :-
```

```
    padre(A, D).
```

```
antepasado(A, D) :-
```

```
    padre(A, Hijo),
```

```
    antepasado(Hijo, D).
```

```
?- antepasado(juan, Quien).
```

```
Quien = ana ;
```

```
Quien = pedro ;
```

```
Quien = luis ;
```

```
false.
```

Caso recursivo primero

```
%! primero_lejos(?A, ?D) is nondet.
```

```
%
```

```
% A es antepasado de D: Las mismas dos cláusulas, en orden inverso.
```

```
primero_lejos(A, D) :-
```

```
    padre(A, Hijo),
```

```
    primero_lejos(Hijo, D).
```

```
primero_lejos(A, D) :-
```

```
    padre(A, D).
```

```
?- primero_lejos(juan, Quien).
```

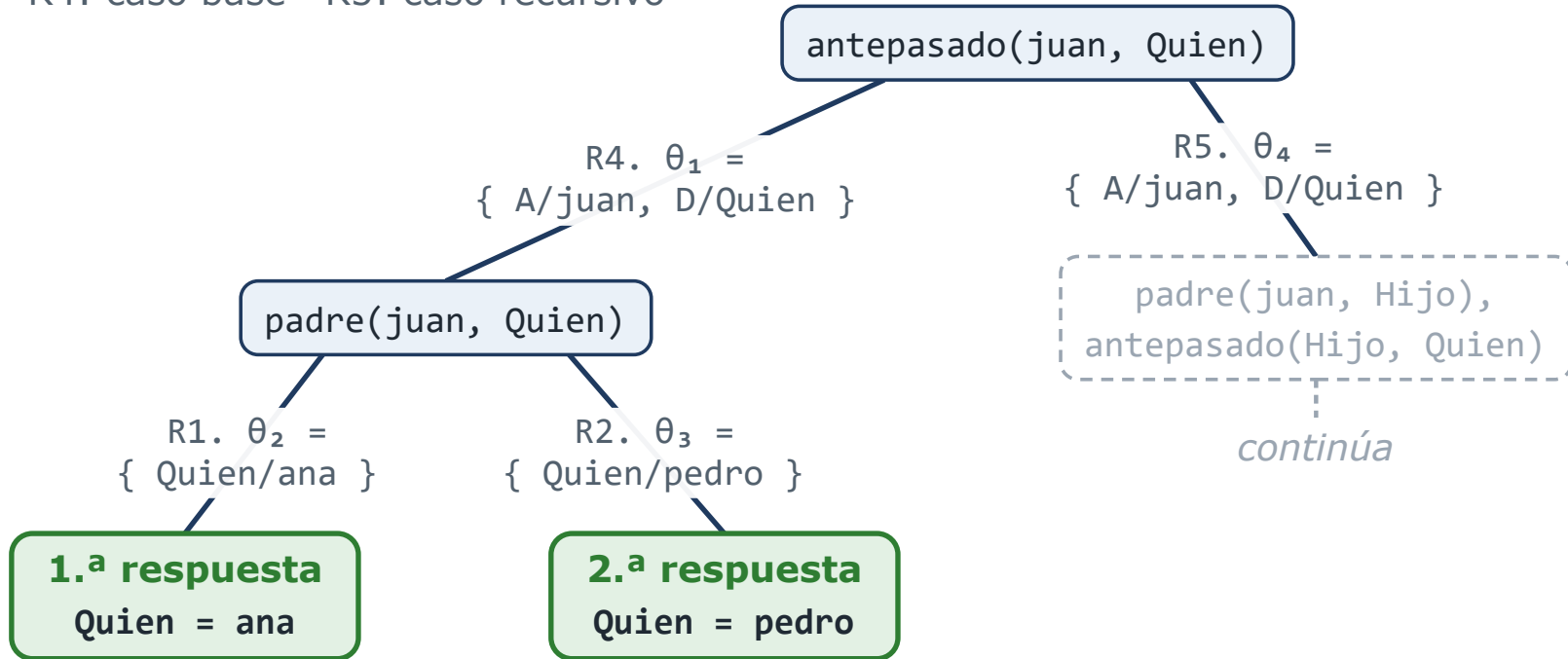
```
Quien = luis ;
```

```
Quien = ana ;
```

```
Quien = pedro.
```

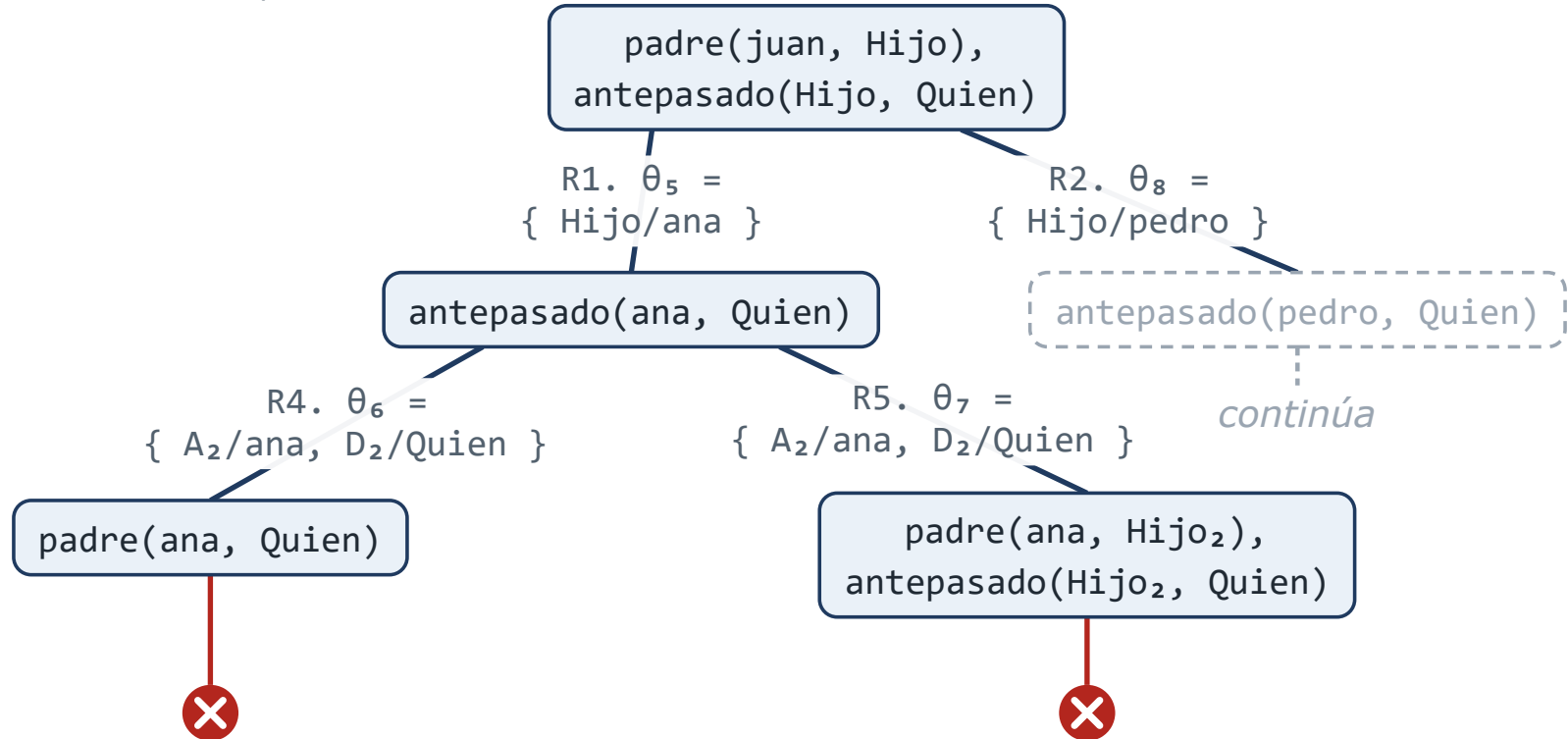
Árbol del caso base primero · 1

R4: caso base · R5: caso recursivo

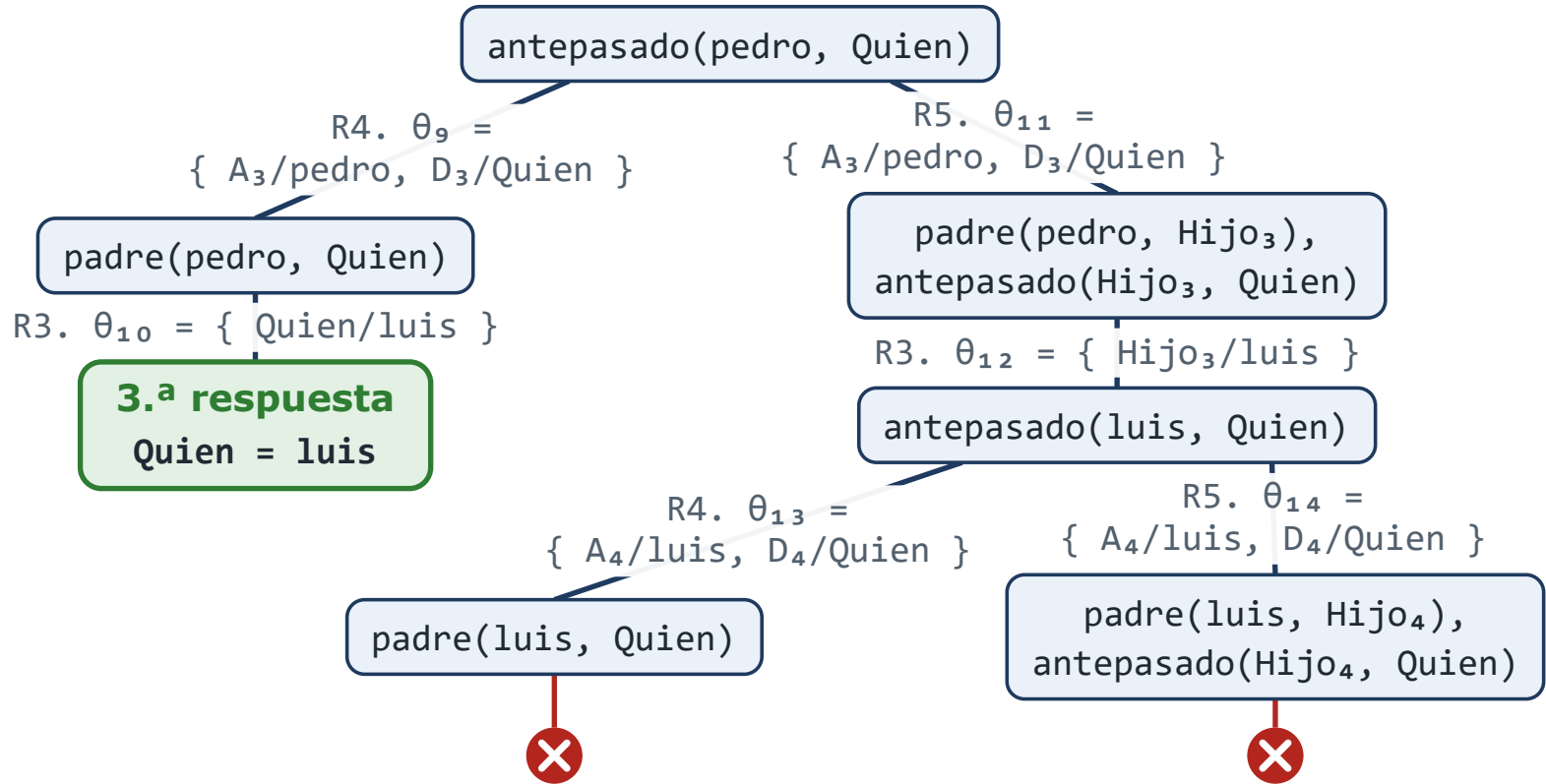


Árbol del caso base primero - 2

la rama de R5, desde θ_4



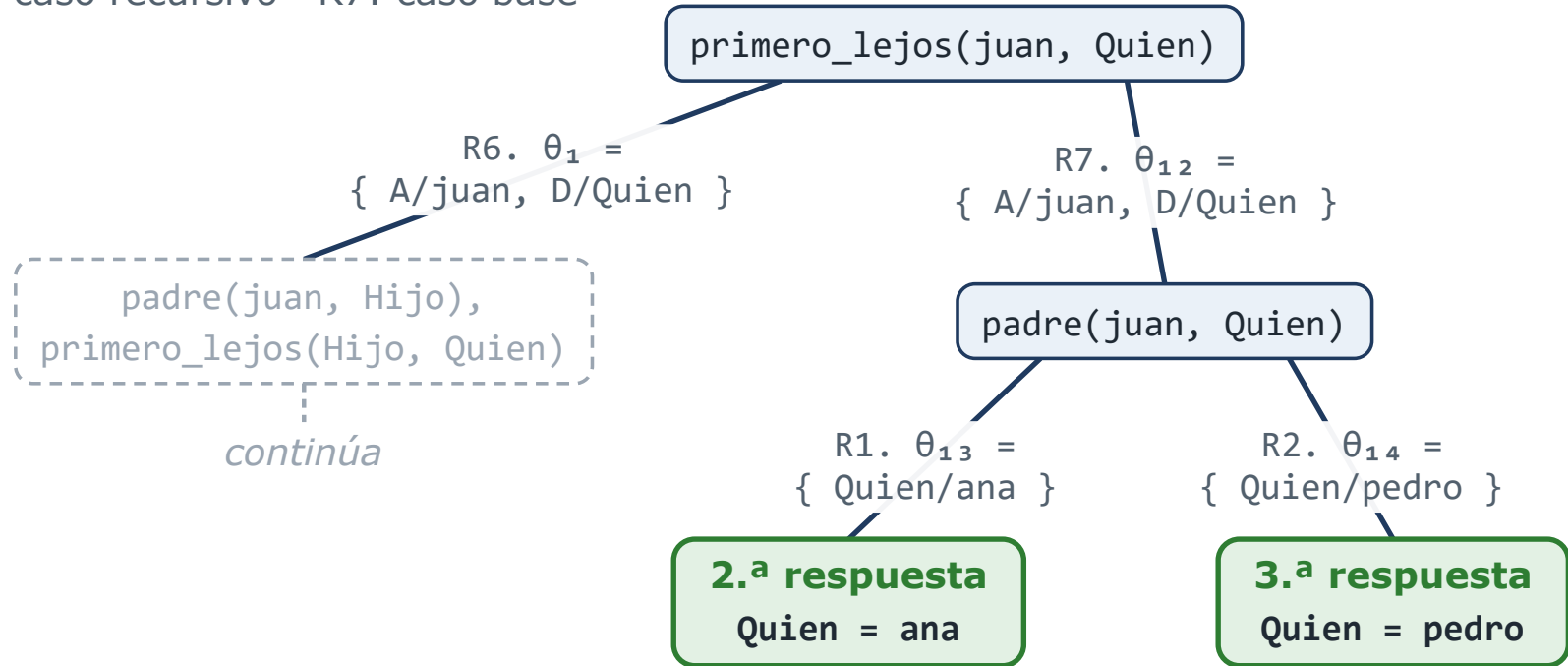
Árbol del caso base primero - 3



la rama de pedro, desde θ_8

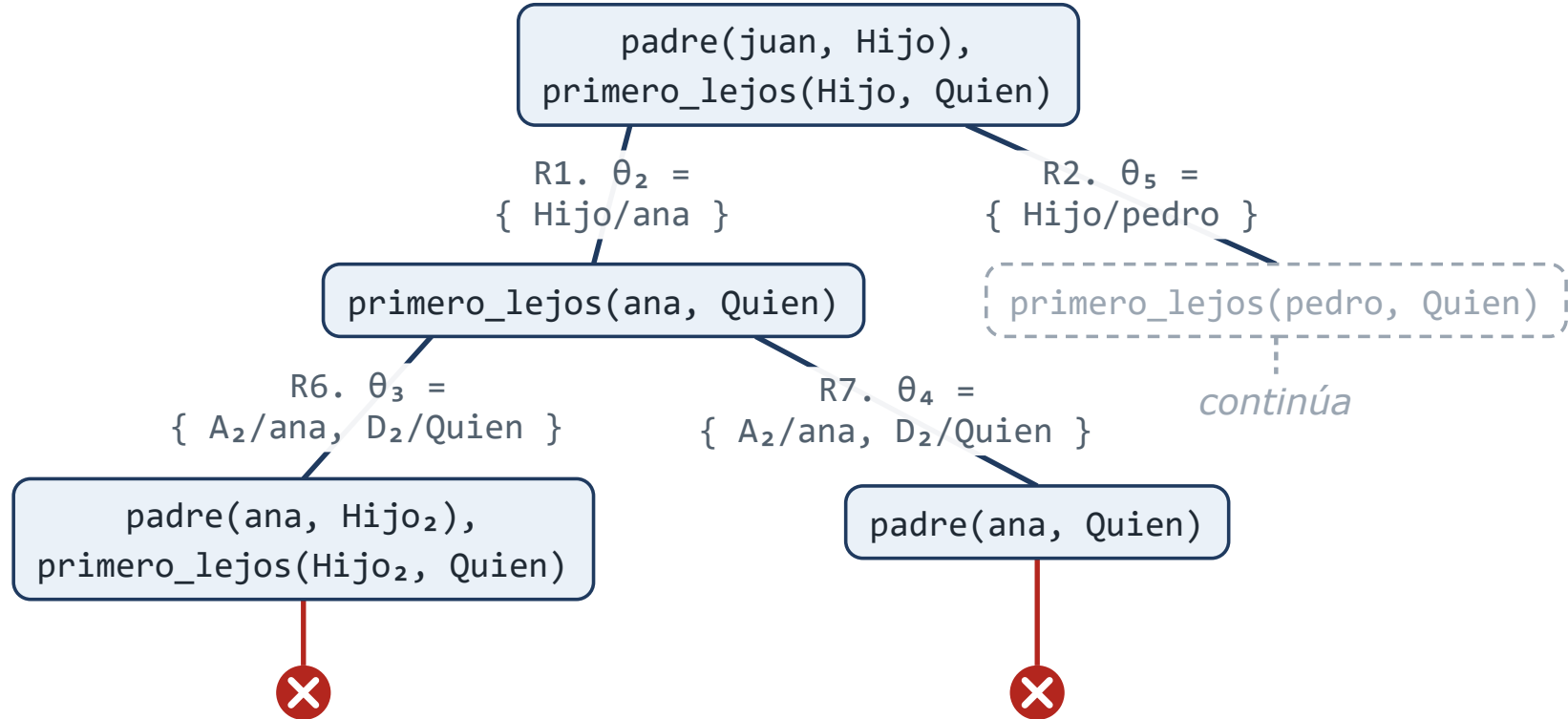
Árbol del caso recursivo primero · 1

R6: caso recursivo · R7: caso base

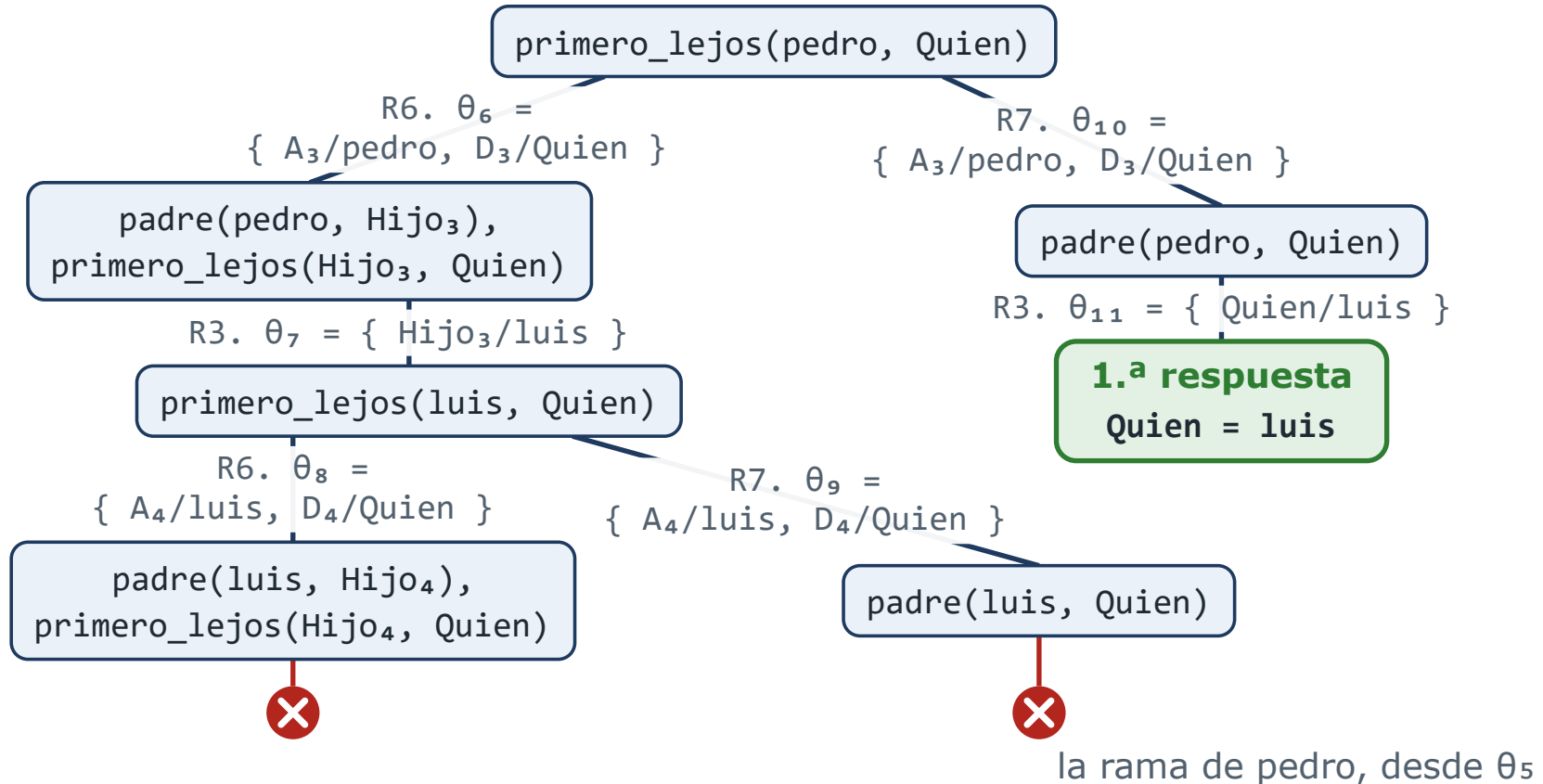


Árbol del caso recursivo primero · 2

la rama de R6, desde θ_1



Árbol del caso recursivo primero · 3

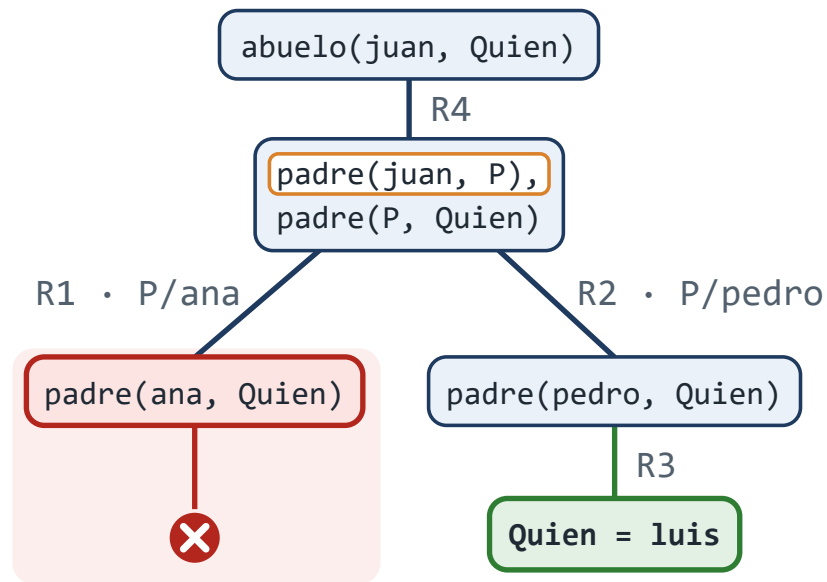


Los objetivos al revés

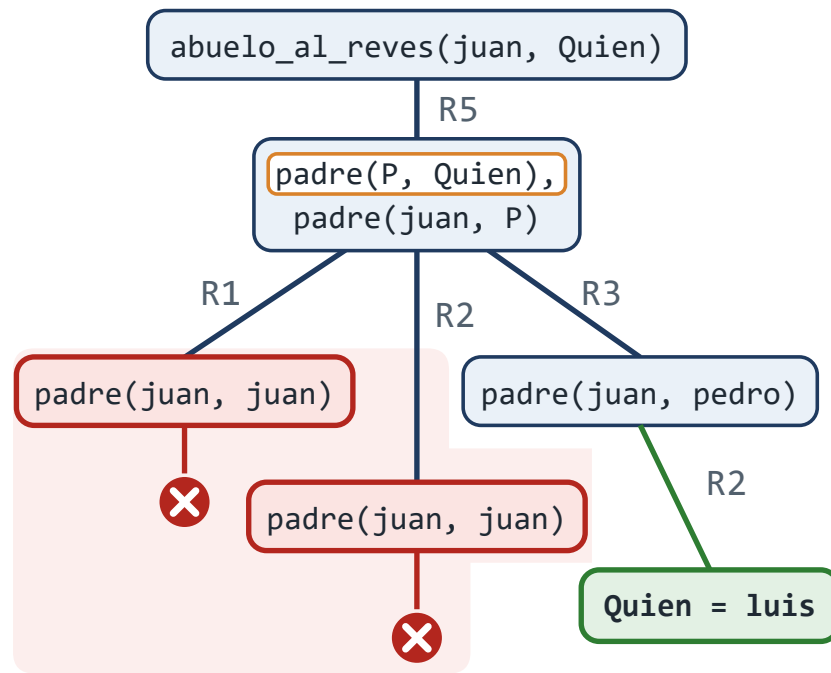
```
abuelo_al_reves(A, N) :-  
    padre(P, N),  
    padre(A, P).
```

```
?- abuelo_al_reves(juan, Quien).  
Quien = luis.
```

Distinto trabajo



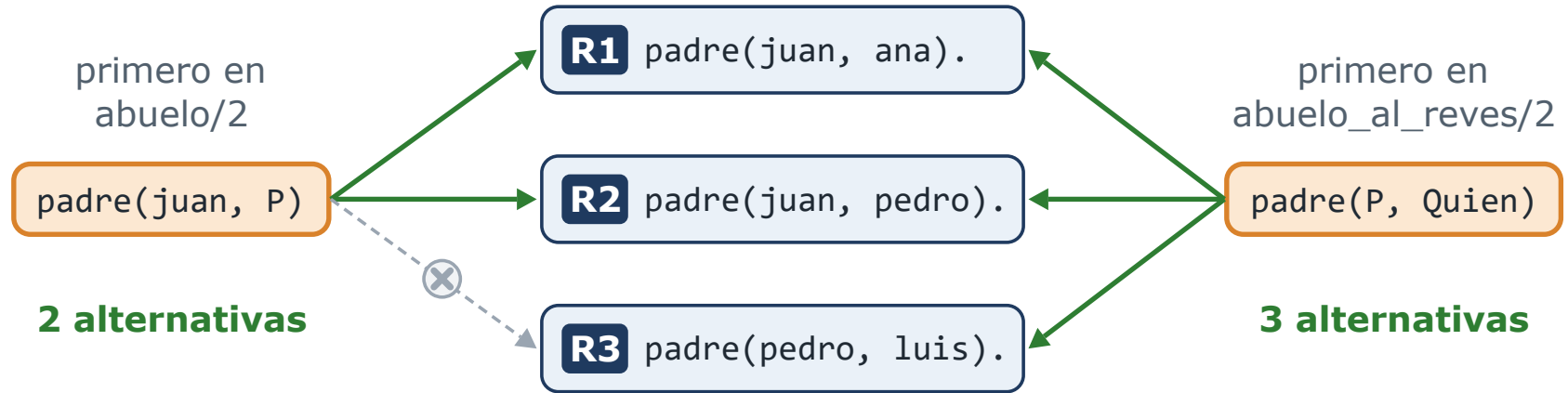
2 ramas 1 hoja de falla



3 ramas 2 hojas de falla

la misma respuesta, distinto trabajo

Menos alternativas primero



primero, el objetivo que genera menos alternativas

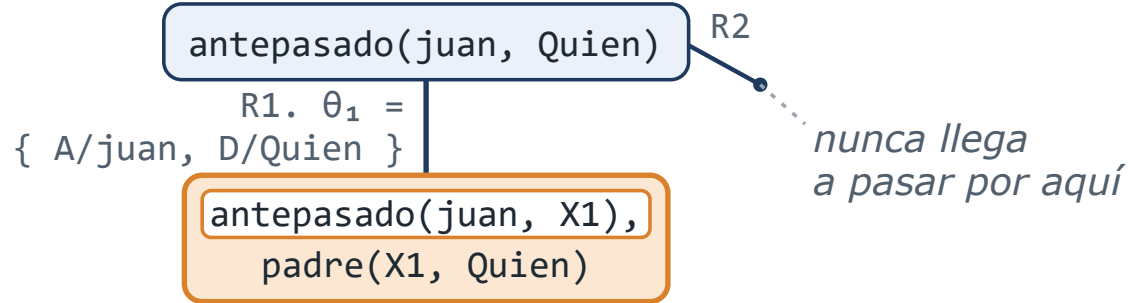
Una versión que no termina

```
%! antepasado(?A, ?D) is nondet.  
%  
% A es antepasado de D. Correcto como afirmación lógica;  
% como programa, no termina.  
antepasado(A, D) :-  
    antepasado(A, X),  
    padre(X, D).  
antepasado(A, D) :-  
    padre(A, D).
```

ERROR: Stack limit (1.0Gb) exceeded

ERROR: Probable infinite recursion (cycle):

Una rama infinita · 1

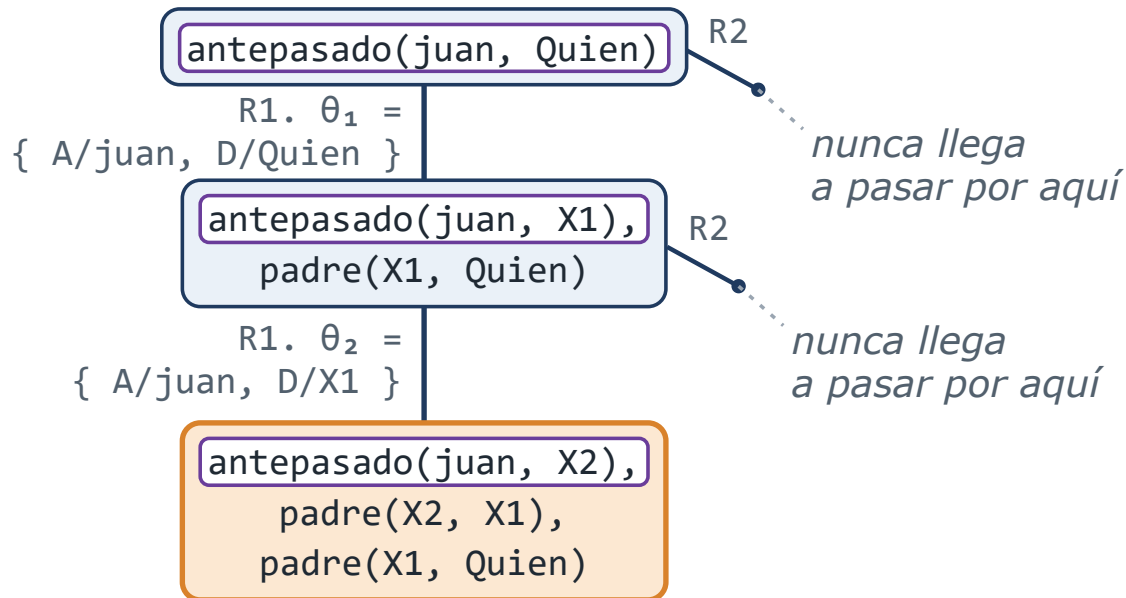


programa

R1 antepasado(A, D) :-
 antepasado(A, X),
 padre(X, D).

R2 antepasado(A, D) :-
 padre(A, D).

Una rama infinita · 2



programa

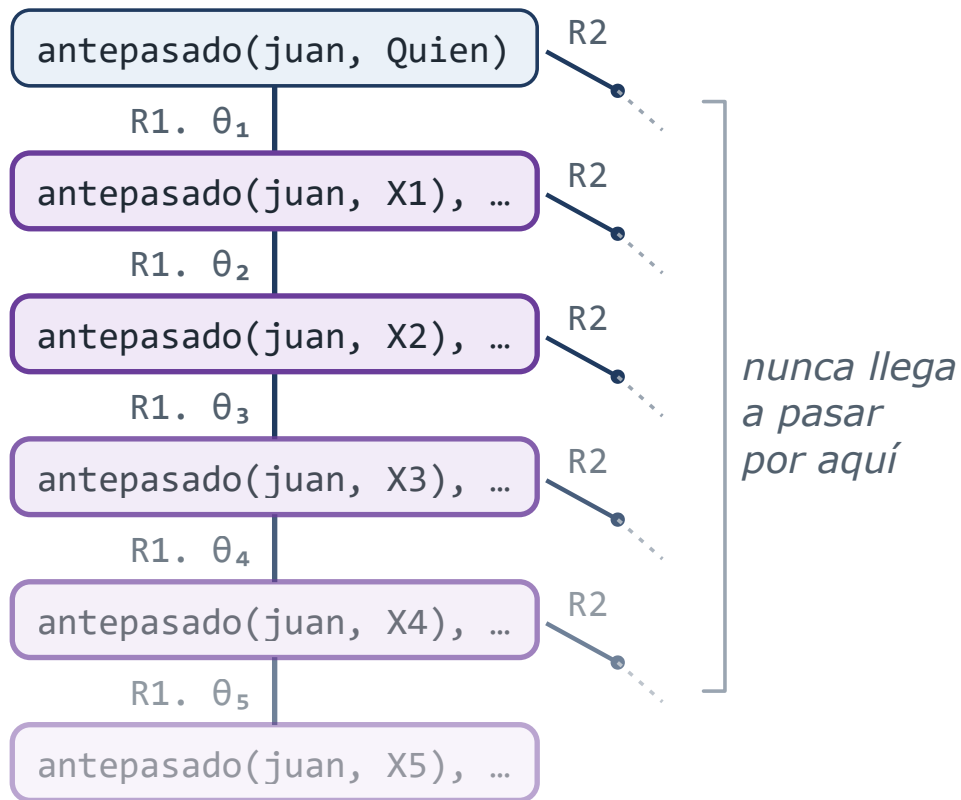
R1 `antepasado(A, D) :-
 antepasado(A, X),
 padre(X, D).`

R2 `antepasado(A, D) :-
 padre(A, D).`

el mismo objetivo

`antepasado(juan, ...)`
con otra variable

Una rama infinita - 3



programa

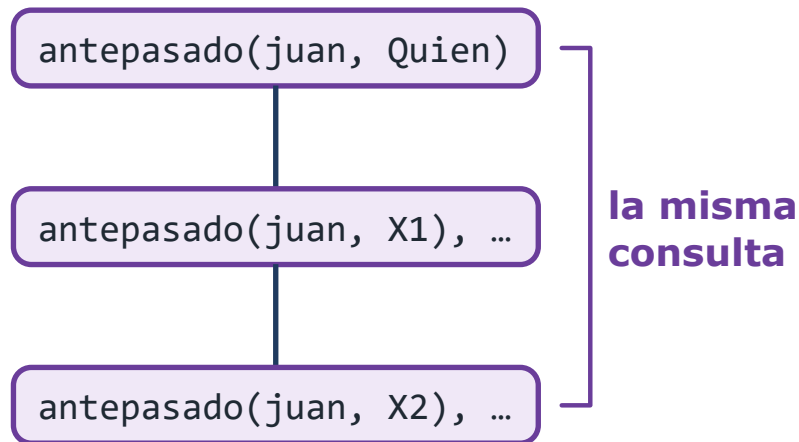
```
R1 antepasado(A, D) :-  
    antepasado(A, X),  
    padre(X, D).
```

```
R2 antepasado(A, D) :-  
    padre(A, D).
```

**ERROR: Stack limit
(1.0Gb) exceeded**

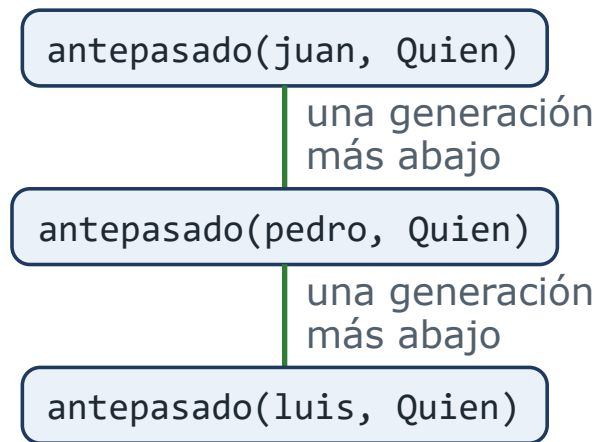
Cómo reconocer una rama infinita

se repite sin avanzar



rama infinita

avanza en cada paso



rama finita

Avanzar antes de la recursión

no termina

```
antepasado(A, D) :-  
    antepasado(A, X),  
    padre(X, D).
```



la llamada recursiva va primero:
la misma consulta, otra vez

termina

```
antepasado(A, D) :-  
    1 padre(A, Hijo),  
    2 antepasado(Hijo, D).
```

primero avanza una generación,
después la llamada recursiva

el objetivo recursivo no se escribe primero

el caso base se escribe primero

Correcto no equivale a ejecutable

recursión primero

antepasado(A, X),
padre(X, D)

avance primero

padre(A, Hijo),
antepasado(Hijo, D)

lectura lógica

lo que afirma



correcto

=



correcto

lectura procedimental

lo que se ejecuta



no termina



termina

el orden no cambia lo que se afirma; decide lo que se ejecuta

Resumen



árbol de derivación

todo lo que Prolog intenta para responder



resolución

el cuerpo en lugar del objetivo, con θ aplicada



orden de los objetivos

cambia el árbol, y con él el trabajo



orden de las cláusulas

cambia el orden de las respuestas



hoja de éxito

una respuesta, de izquierda a derecha



rama infinita

oculta todas las ramas a su derecha